

introduces depolarising noise, offering a more realistic preview of what might happen on actual quantum devices.

Algorithm design: Thinking quantum

Quantum algorithms differ fundamentally from classical ones. They exploit superposition, entanglement, and interference to solve problems in new ways. When designing quantum algorithms, consider:

Parallelism: Qubits can represent multiple states, enabling massive parallel computation.

Entanglement: Qubits can become correlated in ways impossible for classical bits, allowing unique operations.

Measurement: Reading a qubit collapses its state, so algorithms must be carefully structured to get useful results.

Classic examples of quantum algorithms include Shor's algorithm for factoring, Grover's search, and the Deutsch–Jozsa algorithm. Even simple routines like the quantum random number generator highlight the unique logic needed in quantum programming.

Debugging and error handling in quantum code

Debugging quantum programs is fundamentally different—and significantly more challenging—than debugging classical software. In the classical world, a developer can pause execution, inspect the values of variables using a print statement or a debugger, and resume. In the quantum realm, the laws of physics intervene. Due to the 'observer effect', measuring a qubit to check its state collapses its superposition to a definite 0 or 1, effectively destroying the quantum information you intended to inspect. This makes the standard 'breakpoint and watch' approach impossible on real hardware.

The challenge of 'blind' execution

When running on actual quantum processing units (QPUs), the code executes as a 'black box'. You prepare the state, run the circuit, and get a probabilistic distribution of results. If the output is wrong, you cannot easily determine if the error stems from a logic bug in your algorithm or quantum noise (decoherence and gate errors) inherent to the hardware.

To address this, researchers and developers currently rely on a multi-layered approach to debugging.

Classical simulation and state vector inspection: The first line of defence is the simulator. Since simulators (like Qiskit's Aer or Cirq's simulator) run on classical hardware, they are not bound by quantum collapse. They allow developers to peek at the 'state vector'—the mathematical description of the qubits' amplitudes—at any point in the circuit without destroying it. Tools like the Qiskit debugger or IDE extensions allow for 'snapshotting' the state vector to visualise the Bloch sphere (a geometric representation of qubit states) step-by-

step. This verifies the logic of the algorithm in an ideal, noise-free environment.

Quantum assertions: Just as classical code uses 'assert' to verify conditions, quantum programming is adopting quantum assertions. These are specific checks designed to verify properties of a quantum state without fully collapsing it. For example, if an algorithm dictates that two qubits should be entangled, a developer can apply a specific measurement that checks parity without revealing the individual values. If the assertion fails, the program halts, signalling a logic error.

Noise modelling and error mitigation: Once logic is verified, the challenge shifts to hardware errors. Noisy Intermediate-Scale Quantum (NISQ) devices are prone to bit-flips (0 turning into 1) and phase-flips. To debug this, developers use noise models. By importing real calibration data from a backend (e.g., FakeProvider in Qiskit), one can simulate how the circuit should behave on specific noisy hardware.

If the simulation matches the real hardware output, the issue is noise, not logic. To handle this, we employ quantum error mitigation techniques. Unlike full error correction (which requires thousands of physical qubits to make one logical qubit), mitigation involves running the circuit multiple times with slight variations (such as zero noise extrapolation) to mathematically estimate and subtract the noise from the result. This statistical approach is currently the standard for extracting accurate results from imperfect machines.

Practical exercises: Writing your first quantum program

Let's walk through how to build and run a basic quantum coin toss in both Qiskit and Cirq.

Quantum coin toss in Qiskit

```
from qiskit import QuantumCircuit, Aer, execute
# Step 1: Create quantum circuit
qc = QuantumCircuit(1, 1)
# Step 2: Apply Hadamard gate to create 50/50 superposition
qc.h(0)
# Step 3: Measure the qubit
qc.measure(0, 0)
# Step 4: Simulate and print results
simulator = Aer.get_backend('qasm_simulator')
result = execute(qc, simulator, shots=100).result()
counts = result.get_counts(qc)
print("Heads:", counts.get('0', 0), "Tails:", counts.get('1', 0))
```

The sample output from the quantum simulator is:

```
Heads: 46 Tails: 54
```